

DSL for Dynamic Window Management

The idea is to design a DSL that would describe how windows are positioned and sized within a window manager.

I've been thinking about the approach as a context free grammar. Variables are assigned to a combination of other variables, with unique operators describing how the available space is partitioned. For example, most CFGs have a starting variable S . S can be thought of as the entire screen space. We could partition S like so: " $S = \text{full}$ ". For this example we say essentially say "the screen space should be taken up in full by a window".

We can get more creative that this though. If we were to instead define " $S = \text{full } [||] \text{ full}$ ". We would split the available space given to S vertical, giving one window each side. The CFG style of this comes in when we add more variables.

$$\begin{aligned} S &= \text{full } [||] V \\ V &= \text{full } [-] \text{full} \end{aligned}$$

The above example shows how we could we give 1 window the left half of the available space, and two windows split the right half horizontally. This can be extended to say

I think the idea can best be explained in a few examples so below I'll show some of those with small explanations for how the space is partitioned.

$$\begin{aligned} S &= \text{full } [||] V \\ V &= \text{full } (-) (V)? \end{aligned}$$

Here we see some recursion, and some extra goodies to make it a little nicer. First $(-)$ allows us to divide the available space evenly, distributing available space to its children evenly too. This is different from $[-]$, which always splits in half, and gives child operands exactly half of it's available space no matter what. In this case the recursion may also be thought of as " $V = \text{full } (-) \text{full } (-) \dots (-) \text{full}$ ". Finnally we see this $?$ indicating an optional. This simply indicates that if there is no window to fill the space, the entire operator collapses to the other non-optional. So in this case, if no window exists to fill the final variable V we just treat V like the left operand, in this case full . If you are familiar with the common "master/stack" layout, that's what we describe here.

$$\begin{aligned} S &= M [||] V? \\ M &= \text{full } \{ \} M? \\ V &= \text{full } \{v\} V? \end{aligned}$$

This is simmlar to the master/stack layout we described before, but we use a new operator. $\{ \}$ gives each operand it's entire available space, and is able to do so by layering them on top of each other. This gives the effect of the common monocle layout. $\{v\}$ on the other hand, can be thought of as identical to $\{ \}$, but positions the second operand below the first. effectively sequencing the operands in some direction. In a smaller example, if we define " $S = \text{full } \{v\} \text{full } \{>\} \text{full}$ " where we have 3 windows all sized with the available space given to S with a full below the left-most full and another window positioned to the right of that. In the example above, we can think of this being how scrolling window managers like niri behave.